

Week 2 Day 4 Hands-on Lab: Compose Web, DB, Network, Volume

목적

이 문서는 Day 4의 전체 실습을 하나의 실행 흐름으로 묶는다. 교시별 lesson은 개념과 판단 기준을 설명하고, 이 lab guide는 실제 명령, 기대 출력, failure drill, cleanup audit을 제공한다.

Day 4 실습은 네 가지 질문을 확인한다.

1. Day 3의 긴 `docker run` 옵션을 `compose.yaml`로 어떻게 옮기는가?
2. `web`과 `db`는 어떤 `network`와 `service name`으로 연결되는가?
3. `.env`, `volume`, `healthcheck`는 어떤 운영 위험을 줄이는가?
4. 실패했을 때 `config`, `port`, `env`, `volume`, `readiness` 중 어디를 먼저 볼 것인가?

Phase A: 실습 준비와 config 검증

```
cd week2/day4/labs/compose-app
cp .env.example .env
docker compose config
```

기대 결과:

```
services:
  db:
  web:
networks:
  app-net:
volumes:
  pgdata:
```

`docker compose config` 는 container를 실행하기 전에 YAML, variable interpolation, service 구조를 확인하는 사전 점검이다.

Phase B: web + db 실행

```
docker compose up -d
docker compose ps
```

기대 결과:

```
web-1    running
db-1     running (healthy)
```

Docker Desktop 또는 Compose 버전에 따라 출력 형식은 다를 수 있다. 핵심은 `web`이 `running`이고 `db`가 `healthy` 또는 정상 실행 상태라는 점이다.

Phase C: HTTP와 DB evidence

```
curl -I http://localhost:18084
curl -s http://localhost:18084 | grep compose-site-v1
```

```
docker compose logs db
docker compose run --rm db-client
```

기대 결과:

```
HTTP/1.1 200 OK
compose-site-v1
database system is ready to accept connections
current_database | current_user
paperclip        | paperclip
```

db-client 는 같은 Compose network 안에서 `-h db` 로 접속한다. 여기서 `db` 는 container IP가 아니라 Compose service name이다.

Phase D: service name 확인

```
docker compose run --rm db-client pg_isready -h db -U paperclip -d paperclip
```

기대 결과:

```
db:5432 - accepting connections
```

host terminal에서 `db` 라는 DNS 이름이 풀리는 것이 아니다. 같은 Compose network에 붙은 container가 service name을 해석한다.

Phase E: missing env failure drill

`.env` 에서 `POSTGRES_PASSWORD` 줄을 임시로 제거한 뒤 확인한다.

```
docker compose config
```

기대 실패:

```
set POSTGRES_PASSWORD in .env
```

복구:

```
cp .env.example .env
```

실제 수업에서는 `.env` 를 직접 열어 password를 로컬 실습용 값으로 바꾼 뒤 계속 진행한다.

Phase F: wrong port drill

```
curl -I http://localhost:80
curl -I http://localhost:18084
docker compose ps
```

비교할 것:

- host 80이 비어 있으면 실패할 수 있다.
- Compose file의 `ports` 는 기본값으로 host 18084를 web container 80에 연결한다.
- container 내부 nginx port가 아니라 host 접근 port를 먼저 확인한다.

Phase G: cleanup audit

```
docker compose down
docker compose ps
docker volume ls --filter name=compose-app
```

volume까지 삭제하는 실습 cleanup:

```
docker compose down -v
docker volume ls --filter name=compose-app
```

주의:

- `down` 은 container와 network를 정리한다.
- `down -v` 는 named volume까지 삭제한다.
- DB data를 보존해야 하는 환경에서는 `down -v` 를 runbook 기본값으로 두지 않는다.

기록 템플릿

Day 4 Compose Evidence

- Compose file:
- Config check:
- Project name:
- Web service:
- DB service:
- Host port:
- HTTP status:
- Body marker:
- DB health:
- Service name connection:
- Volume:
- Failure drill:
- Cleanup:
- ``down`` vs ``down -v`` 차이:

Phase H: config output 해석

`docker compose config` 는 긴 출력이므로 전부 외우지 않는다. 아래 항목만 찾아 표시한다.

```
docker compose config | sed -n '1,120p'
```

찾을 항목:

항목	의미
<code>services.web.ports</code>	host 18084가 container 80으로 publish됨
<code>services.db.environment</code>	DB 초기화에 필요한 env 값
<code>services.db.healthcheck</code>	readiness 확인 명령
<code>networks.app-net</code>	service 간 통신 network
<code>volumes.pgdata</code>	PostgreSQL data persistence

기록 예시:

```
Config에서 web published port는 18084이고, db healthcheck는 pg_isready를 사용한다.
```

Phase I: logs 관찰

DB가 healthy가 되지 않거나 query가 실패하면 logs를 본다.

```
docker compose logs db
```

볼 문장:

```
database system is ready to accept connections
```

이 문장은 PostgreSQL server process가 connection을 받을 준비가 됐다는 의미다. 하지만 application 전체가 정상이라는 뜻은 아니므로 query evidence를 추가로 남긴다.

Phase J: network boundary 확인

host와 Compose network의 차이를 확인한다.

```
docker compose run --rm db-client pg_isready -h db -U paperclip -d paperclip
```

기대 결과:

```
db:5432 - accepting connections
```

비교 설명:

위치	접근 대상	설명
host terminal	localhost:18084	web의 published host port
Compose container	db:5432	service name 기반 DB 접근
host terminal	db:5432	일반적으로 실패 또는 해석 안 됨

Phase K: volume lifecycle 확인

```
docker volume ls --filter name=compose-app
```

`docker compose down` 후에도 volume이 남을 수 있다. `down -v` 를 실행하면 Compose project의 named volume까지 삭제된다.

기록할 문장:

```
`docker compose down`은 container와 network 중심 cleanup이고, `down -v`는 DB data volume 삭제를 포함한다.
```

Phase L: Short RCA 작성

failure drill 중 하나를 골라 짧게 작성한다.

```
## Short RCA
- Failed command:
```

- Error line:
- Category:
- Hypothesis:
- Fix:
- Recheck:
- Prevention:

예시:

```
## Short RCA
- Failed command: curl -I http://localhost:80
- Error line: connection failed
- Category: wrong port
- Hypothesis: web service is published on host 18084, not 80
- Fix: use http://localhost:18084
- Recheck: HTTP/1.1 200 OK
- Prevention: README에 host port를 명시한다
```

Phase M: README handoff 작성

실습 종료 전 README 또는 개인 노트에 다음 section을 넣는다.

```
## Docker Compose
### Setup
```bash
cp .env.example .env
```

### Run
```bash
docker compose config
docker compose up -d
```

### Check
```bash
docker compose ps
curl -I http://localhost:18084
docker compose run --rm db-client
```

### Cleanup
```bash
docker compose down
```

실습 DB data까지 초기화할 때만:

```bash
docker compose down -v
```
```

Troubleshooting Matrix

| 증상 | 먼저 볼 것 | 다음 행동 |
|----|--------|-------|
|----|--------|-------|

| | | |
|-------------------------|---------------------|----------------------|
| POSTGRES_PASSWORD error | .env, config | .env.example 복사 후 수정 |
| web 접속 실패 | compose ps, port | host port 18084 확인 |
| DB query 실패 | logs db, health | DB readiness 확인 |
| init SQL 반영 안 됨 | named volume | 실습 reset이면 down -v |
| service name 실패 | network, command 위치 | container 안에서 db 사용 |
| cleanup 실패 | 남은 container | compose ps -a 확인 |

Safety Rules

- 실제 운영 secret을 .env.example 에 쓰지 않는다.
- 공개 repository에 .env 를 commit하지 않는다.
- down -v 실행 전 삭제되는 data가 실습 data인지 확인한다.
- port 충돌은 host port 변경으로 해결한다.
- DB port를 host에 열 필요가 없으면 열지 않는다.
- 실패한 명령과 출력은 지우지 말고 RCA에 남긴다.

Strong Evidence Checklist

- ☐ docker compose config output에서 services/networks/volumes 확인
- ☐ docker compose ps 에서 web/db 상태 확인
- ☐ curl -I http://localhost:18084 에서 HTTP 200 확인
- ☐ grep compose-site-v1 body marker 확인
- ☐ docker compose run --rm db-client query 확인
- ☐ missing env 또는 wrong port drill 수행
- ☐ down 과 down -v 차이 문서화
- ☐ cleanup 후 남은 container/volume 확인

Self-Assessment

| 항목 | 0 | 1 | 2 |
|---------|-------|---------------|---------------------------|
| Config | 실행 못함 | 실행만 함 | 출력 구조 설명 |
| Web | 확인 없음 | running만 확인 | HTTP와 body marker |
| DB | 확인 없음 | healthy만 확인 | query evidence |
| Network | 모름 | network 이름만 앎 | service name db 설명 |
| Volume | 모름 | volume 이름만 앎 | cleanup 위험 설명 |
| RCA | 없음 | 증상만 기록 | fix/recheck/prevention 포함 |

Lab Completion

이 lab은 다음 문장을 evidence와 함께 말할 수 있을 때 완료된다.

나는 web과 db를 Compose project로 실행했고, host에서는 localhost:18084로 web을 확인했으며, Compose network 안에서는 service name db로 PostgreSQL에 접근했다. cleanup에서는 down과 down -v의 차이를 구분했다.